A PROJECT REPORT

AI & Machine Learning Internship Projects

Six Foundational ML Tasks and a Disease Outbreak Prediction Pipeline

submitted in partial fulfilment of the requirements of the
AICTE Internship on AI: Transformative Learning
with **TechSaksham** - a joint CSR initiative of Microsoft & SAP

by

ISHA LOHANI

Under the guidance of
Mr. P Raja

October 2026

Acknowledgement

I would like to express my sincere gratitude to my guide, Mr. P Raja, for his guidance and encouragement throughout this internship. I also thank AICTE and TechSaksham, a joint CSR initiative of Microsoft and SAP, for providing the opportunity and platform to learn Artificial Intelligence and Machine Learning through hands-on work.

I am grateful to my peers for their feedback, and to my family and friends for their constant support.

Isha Lohani

Abstract

Machine learning is only useful when models are built on sound data, evaluated honestly and packaged so that others can reproduce the work. This report documents a portfolio of seven projects completed during an AI/ML internship: six foundational tasks (data preprocessing, exploratory data analysis, linear regression, logistic regression, decision trees with random forests, and K-nearest neighbours) and a larger capstone, a **disease outbreak prediction pipeline**.

The foundational tasks use real, publicly available datasets (Titanic, Diabetes, Breast Cancer Wisconsin, UCI Heart Disease and Iris). The logistic regression model reaches 98.25% accuracy and 0.995 ROC-AUC on breast-cancer diagnosis, and a random forest outperforms a single decision tree on heart-disease classification (86.7% vs 80.0% test accuracy). The capstone predicts whether a region will experience an outbreak in the following week from weather, demographic and surveillance features. Because real outbreak surveillance data is not freely available in a clean form, the capstone uses a transparent, reproducible **simulated** dataset; its results demonstrate the machine-learning workflow rather than real epidemiology.

The report also records the quality-assurance work carried out on the code: replacing synthetic data with real data, correcting results that did not match the code, adopting leakage-free pipelines and time-based evaluation, comparing models against naive baselines, and verifying that every script runs end to end.

Table of Contents

Acknowledgement 2

Abstract 2

Table of Contents 3

1. Introduction 4

1.1 Problem Statement . . . 4

1.2 Motivation . . . 4

1.3 Objectives . . . 4

1.4 Scope . . . 4

2. Literature Survey 5

2.1 Review of Relevant Literature . . . 5

2.2 Existing Models and Techniques . . . 5

2.3 Gaps Addressed by This Work . . . 5

3. Proposed Methodology 6

3.1 System Design . . . 6

3.2 Datasets . . . 6

3.3 Requirement Specification . . . 6

4. Implementation and Results 7

4.1 Foundational Tasks . . . 7

4.2 Capstone: Disease Outbreak Prediction . . . 10

4.3 Verification and Reproducibility . . . 13

5. Discussion and Conclusion 14

5.1 Key Lessons . . . 14

5.2 Limitations . . . 14

5.3 Future Work . . . 14

5.4 Conclusion . . . 14

References 15

Appendix A: Repository Structure 15

1. Introduction

1.1 Problem Statement

Learning machine learning from tutorials often leaves a gap between running a notebook and building something trustworthy. Typical weaknesses include evaluating a model on data it has effectively seen, reporting accuracy without a baseline, using artificial data that guarantees good results, and publishing code that nobody else can run. This work addresses those weaknesses by building each model carefully and verifying it.

The capstone addresses a public-health question: *given this week's conditions in a region, will an outbreak occur next week?* Early warning allows health agencies to allocate resources before a surge in cases.

1.2 Motivation

Disease outbreaks such as seasonal influenza, dengue and COVID-19 affect health systems and economies. Data-driven early-warning models are an active research area, and building one end to end is an effective way to practise the full ML workflow: data preparation, modelling, validation, interpretation and deployment.

1.3 Objectives

- Apply core ML techniques (preprocessing, EDA, regression, classification, tree ensembles, instance-based learning) to real datasets.
- Build an outbreak-prediction pipeline using leakage-free preprocessing and time-aware validation.
- Evaluate every model with appropriate metrics and compare against simple baselines.
- Make all results reproducible: one command runs every project and every figure is regenerated from code.

1.4 Scope

The work covers seven projects implemented in Python with scikit-learn. The capstone is a weekly, region-level, single-step forecast on simulated data. It does not model spatial spread between regions, use real-time feeds, or claim clinical or public-health validity.

> **Important note on data.** The capstone dataset is simulated. Every number reported for it describes the behaviour of models on that simulation, not performance on real outbreaks.

2. Literature Survey

2.1 Review of Relevant Literature

Mathematical models of infectious disease have a long history, beginning with compartmental models that divide a population into susceptible, infected and recovered groups (Anderson & May, 1991). The COVID-19 pandemic produced a large body of work on transmission dynamics and intervention effects (Kucharski et al., 2020; Chinazzi et al., 2020). In parallel, machine learning has become widely used in medicine for prediction and decision support (Rajkomar et al., 2019).

2.2 Existing Models and Techniques

Traditional approaches use SIR/SEIR-type compartmental models, which encode biological assumptions explicitly. Machine-learning approaches instead learn patterns from historical data using logistic regression, tree ensembles and neural networks. Tree ensembles such as random forests and gradient boosting are popular for tabular data because they handle non-linear relationships and mixed feature types with little tuning.

2.3 Gaps Addressed by This Work

Many student projects report a single accuracy figure from a random train/test split. For time-ordered data this is optimistic, because the test set can contain weeks that precede training weeks. This work therefore uses a time-based split and time-series cross-validation, and reports performance relative to a naive persistence baseline so that the genuine added value of the model is visible.

3. Proposed Methodology

3.1 System Design

All projects follow the same workflow, summarised below. The capstone additionally persists the trained model and provides a command-line predictor.

Stage	Description
1. Data	Load a bundled/public dataset (Tasks 1-6) or generate the simulated surveillance dataset (capstone).
2. Preparation	Handle missing values, encode categoricals, scale numeric features. Inside a scikit-learn Pipeline so that statistics are learned from training data only.
3. Split	Stratified random split for tabular tasks; **time-based** split (last year held out) for the capstone.
4. Modelling	Fit one or more models; select among candidates with cross-validation (time-series CV for the capstone).
5. Evaluation	Accuracy, precision, recall, F1, ROC-AUC, confusion matrix, regression error metrics; compare against baselines.
6. Interpretation	Feature importance, coefficients, variance inflation factors, residual plots.
7. Delivery	Figures saved to disk, README per project, saved model and predictor, `run_all.py` smoke test.

Table 1: Common workflow used across the projects.

3.2 Datasets

Project	Dataset	Size	Task
Task 1, 2	Titanic passengers (public CSV)	891 rows in the standard file	Preprocessing / EDA
Task 3	scikit-learn Diabetes	442 patients, 10 features	Regression
Task 4	Breast Cancer Wisconsin	569 samples, 30 features	Binary classification
Task 5	UCI Cleveland Heart Disease	297 patients, 13 features	Binary classification
Task 6	Iris	150 samples, 4 features	3-class classification
Capstone	**Simulated** weekly surveillance	2,600 rows (10 regions x 260 weeks)	Binary forecasting

Table 2: Datasets used.

3.3 Requirement Specification

Hardware: any modern laptop; all experiments run in seconds on a standard CPU with no GPU. **Software:** Python 3, NumPy, pandas, matplotlib, seaborn, scikit-learn and joblib (listed in `requirements.txt`).

4. Implementation and Results

4.1 Foundational Tasks

Task 1 - Data Cleaning and Preprocessing

The Titanic dataset is cleaned in six steps: inspection, missing-value handling (Age filled with the median, Embarked with the mode, Cabin dropped because roughly three quarters of its values are missing), encoding (Sex mapped to 0/1, Embarked one-hot encoded), standardisation of Age and Fare, boxplot-based outlier inspection, and removal of extreme Fare outliers using the 1.5 x IQR rule. The script reads a local `titanic.csv` if present, otherwise downloads and caches it, and saves the cleaned dataset.

Task 2 - Exploratory Data Analysis

The EDA script produces histograms, Age and Fare distributions, boxplots, a pairplot, a correlation heatmap and skewness measures, and analyses survival rate by passenger class and sex. The observations printed at the end of the script are computed from the data rather than hard-coded, so they cannot become out of date.

Task 3 - Linear Regression on the Diabetes Dataset

An earlier version of this task used randomly generated house prices created from a fixed formula, which only demonstrated that the model could recover that formula. It was replaced with the real Diabetes dataset (442 patients; target: disease progression after one year). A simple model using only BMI is compared with a multiple regression using all ten features.

Model	MAE	RMSE	R^2
Simple regression (BMI only)	52.26	63.73	0.233
Multiple regression (10 features)	42.79	53.85	0.453

Table 3: Linear regression results on the 20% test set.

Using all features roughly doubles the explained variance, but R^2 of 0.45 shows that much variation remains unexplained, which is realistic for noisy medical data. A multicollinearity check using variance inflation factors (VIF) shows very high values for the blood-serum measures s1 (59.2) and s2 (39.2), and elevated values for s3 (15.4) and s5 (10.1); individual coefficients for these features are therefore unreliable even though overall predictions remain sound.

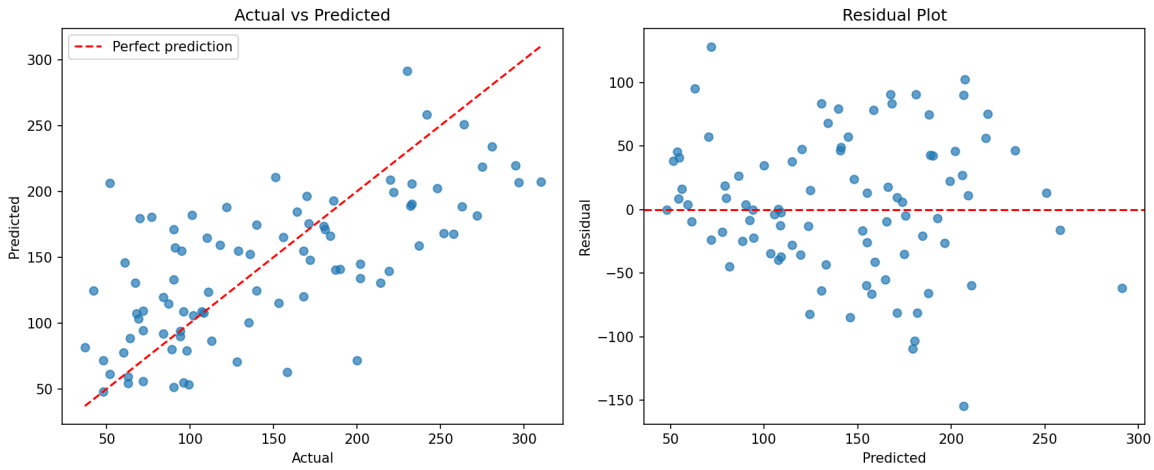


Figure 1: Actual vs predicted values and residual plot for the multiple regression model.

Task 4 - Logistic Regression

A logistic regression classifier is trained to distinguish malignant from benign tumours on the Breast Cancer Wisconsin dataset using a stratified 80/20 split (455 training and 114 test samples) with features standardised on the training set only.

Metric	Result
Accuracy	98.25%
ROC-AUC	0.9954
Precision / recall (malignant)	0.98 / 0.98
Precision / recall (benign)	0.99 / 0.99
Confusion matrix (rows actual; malignant, benign)	[[41, 1], [1, 71]]

Table 4: Logistic regression test results (112 of 114 samples correct).

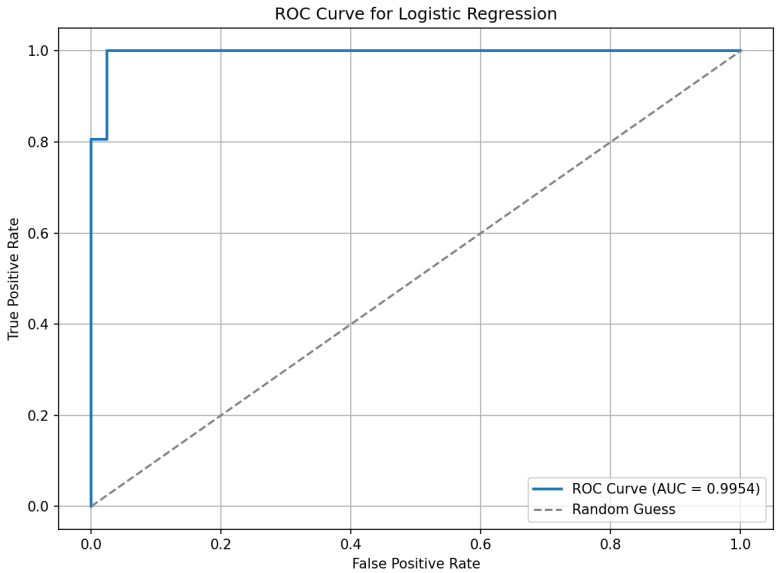


Figure 2: ROC curve of the logistic regression model (AUC = 0.9954).

Task 5 - Decision Tree and Random Forest

Using the UCI Cleveland Heart Disease data (297 patients; 137 with disease, 160 without), a depth-3 decision tree is compared with a 200-tree random forest on a stratified split (237 training, 60 test), followed by 5-fold cross-validation on the training data.

Model	Test accuracy	5-fold CV accuracy
Decision Tree (max depth 3)	80.00%	80.12%
Random Forest (200 trees)	86.67%	82.64%

Table 5: Tree vs forest on heart-disease classification.

The forest is better on both measures. With only 60 test patients, each patient is worth about 1.7 percentage points, so the cross-validation figures are more trustworthy: the real gap is a few points rather than ten. The most important features were *thal*, *cp* (chest-pain type), *thalach* (maximum heart rate), *oldpeak* and *ca*.

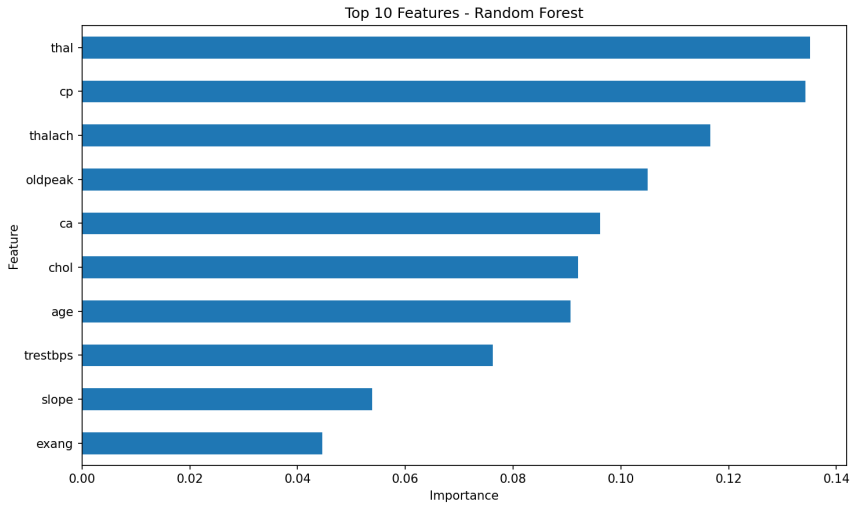


Figure 3: Top-10 random forest feature importances for heart-disease classification.

Task 6 - K-Nearest Neighbours

KNN is applied to the Iris dataset with a stratified 70/30 split (105 training, 45 test) and features scaled using training statistics only, because distance-based methods are sensitive to feature scale.

K	Test accuracy
3	91.11%
5	91.11%
7	93.33%

Table 6: KNN accuracy for different values of K.

K = 7 scores highest, but with 45 test samples the difference is a single flower and is not strong evidence that K = 7 is genuinely better.

4.2 Capstone: Disease Outbreak Prediction

Prediction target

For each region and week, the model predicts `outbreak_next_week`: 1 if the following week's incidence is at least 60 cases per 100,000 people, otherwise 0. The overall outbreak rate in the dataset is 49%.

Simulated dataset

Why simulated: real outbreak surveillance data is not available as one clean, free download, so `generate_data.py` creates a reproducible stand-in (10 regions x 5 years of weekly records, fixed random seed). Weather follows yearly seasonal cycles plus noise. Weekly incidence is auto-regressive on the log scale and is pushed up by humidity, rainfall, mobility and population density, and pushed down by vaccination coverage. About 3% of weather readings are deliberately removed to mimic sensor gaps.

Feature	Meaning
region	Region identifier (categorical)
week_of_year	Position in the yearly cycle (1-52)
temperature_c, humidity_pct, rainfall_mm	Weather conditions this week (some missing)
population_density, vaccination_rate_pct, mobility_index	Demographic and behavioural context
infection_rate, infection_rate_prev	Cases per 100k this week and last week
infection_rate_change	Derived: difference between the two (added in the pipeline)

Table 7: Input features of the outbreak model.

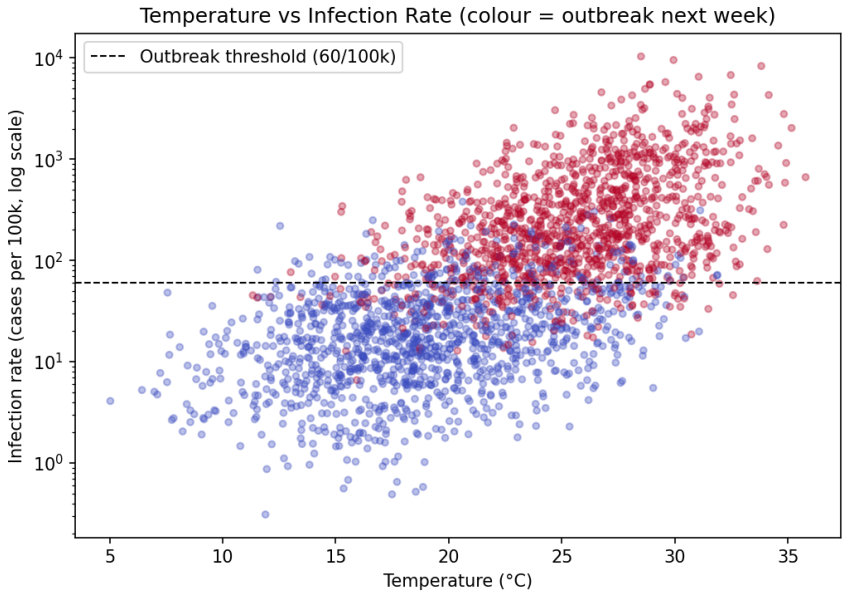


Figure 4: Temperature versus infection rate (log scale) in the simulated data; colour shows whether an outbreak followed the next week.

Modelling pipeline

A single scikit-learn `Pipeline` performs median imputation, standardisation of numeric features, one-hot encoding of the region, and the classifier. This replaces the original forward-fill imputation, which is inappropriate for non-sequential data and no longer works with current pandas. Three classifiers are compared: logistic regression, random forest (300 trees) and gradient boosting.

Evaluation design

- **Time-based split:** weeks 0-207 (four years, 2,080 rows) for training and the final year (520 rows) for testing, so no future information leaks into training.
- **Model selection:** 5-fold `TimeSeriesSplit` cross-validation on training data using ROC-AUC. Random forest scored 0.9716, logistic regression 0.9701 and gradient boosting 0.9699 - effectively tied.
- **Baselines:** (a) always predict the majority class; (b) *persistence* - predict an outbreak next week if the current infection rate is already at or above the threshold.

Results on the held-out year

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Logistic Regression	0.900	0.873	0.896	0.884	0.967
Random Forest (best by CV)	**0.904**	0.881	0.896	0.888	0.967
Gradient Boosting	0.898	0.866	0.901	0.883	0.967
Baseline: persistence	0.879	0.857	0.860	0.858	-
Baseline: majority class	0.427	-	-	-	-

Table 8: Test-set performance (520 rows, final simulated year).

Reading the results honestly. The models beat the persistence baseline by only about 2-3 accuracy points, because outbreaks in this simulation are strongly persistent: a region that is in an outbreak this week usually still is next week. The three models are essentially tied, so the simpler and more interpretable logistic regression is just as defensible a choice as the random forest. A large gain over the majority-class baseline (0.904 vs 0.427) is expected and says little on its own - the persistence baseline is the meaningful comparison.

The random forest's confusion matrix on the test year contains 271 true negatives, 27 false positives, 23 false negatives and 199 true positives. In an early-warning setting false negatives (missed outbreaks) are usually costlier than false alarms, so the decision threshold could be lowered below 0.5 to favour recall.

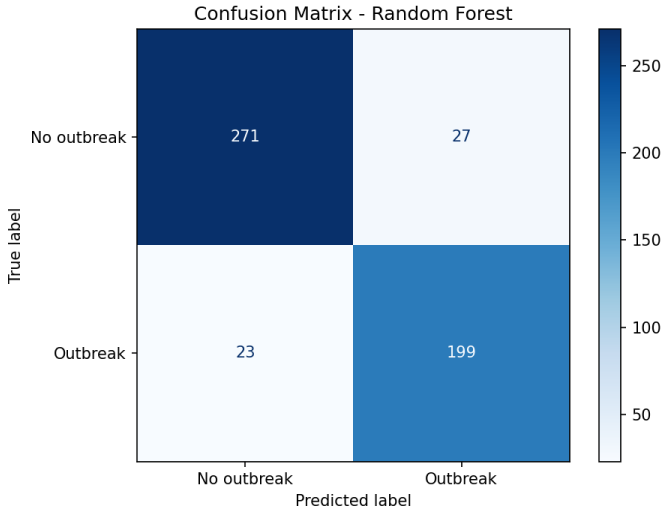


Figure 5: Confusion matrix of the random forest on the held-out year.

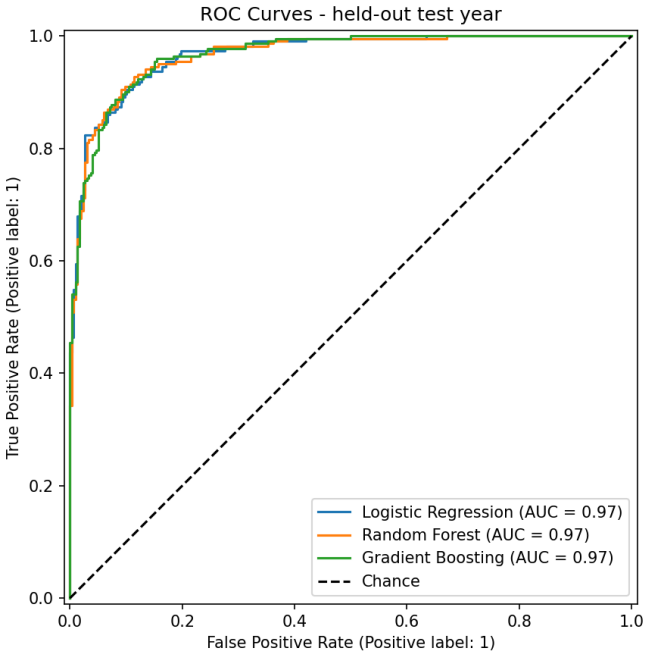


Figure 6: ROC curves of the three models on the held-out year.

Feature importance

Permutation importance (the drop in ROC-AUC when a feature is shuffled) ranks the current infection rate first (0.048), followed by humidity (0.024), the previous week's infection rate (0.012) and rainfall (0.006). Because the relationships were written into the simulator, these rankings reflect the simulation's design and should not be read as epidemiological findings.

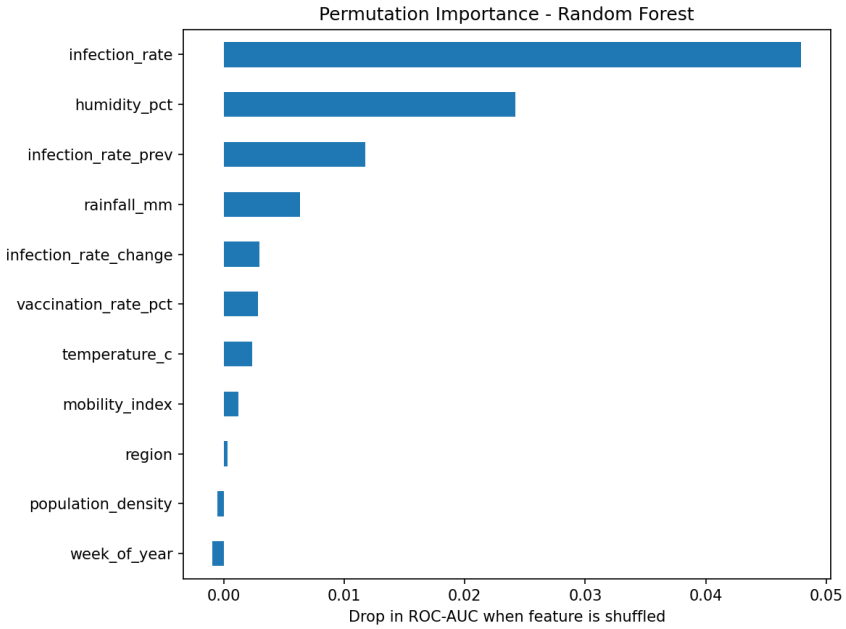


Figure 7: Permutation importance of the random forest on the held-out year.

Deployment: saved model and predictor

The best pipeline is saved with joblib, and `predict.py` loads it to return an outbreak probability for a single region. For example, hot, humid, high-mobility conditions with a rising infection rate give about 80% probability, whereas cool, dry, well-vaccinated conditions give about 0.3%.

```
python main.py                      # train, evaluate, save model + figures + metrics
python predict.py --example high    # predicted outbreak probability: 79.6%
python predict.py --example low     # predicted outbreak probability: 0.3%
```

4.3 Verification and Reproducibility

A review of the original code found issues that prevented it from running or misrepresented its results. All were corrected and re-tested:

Issue found	Resolution
Capstone crashed: dataset file missing; `fillna(method=...)` unsupported in current pandas	Added dataset generator and a scikit-learn imputation pipeline
Task 3 used randomly generated prices; R^2 reflected the generating formula	Replaced with the real Diabetes dataset; added VIF and residual analysis
Task 5 README reported 75% / 85%; code produced 80.0% / 86.7%	README updated to match actual output, plus cross-validation figures
Hard-coded 'observations' in EDA; Windows-only paths; stray backup files; nested folders	Observations computed from data; paths removed; repository restructured
No way to confirm everything works	`run_all.py` runs every script and prints PASS / FAIL; verified on a fresh copy of the repository

Table 9: Problems found during review and how they were resolved.

All scripts run successfully. Regenerating the seeded simulated data from a clean copy gave a byte-identical file and identical metrics.

5. Discussion and Conclusion

5.1 Key Lessons

- **Evaluation design matters as much as the model.** A time-based split and a persistence baseline changed the story from 'the model is 90% accurate' to 'the model adds a few points over a simple rule'.
- **Real data keeps you honest.** Replacing synthetic house prices with real medical data lowered R^2 from about 0.99 to 0.45 - a truthful result.
- **Small test sets are noisy.** With 45-60 test samples, one prediction moves accuracy by 1.7-2.2 points; cross-validation gives a steadier comparison.
- **Reproducibility is a feature.** Saving figures, fixing seeds and providing a one-command smoke test makes the work verifiable.

5.2 Limitations

- The capstone uses simulated data; its relationships are those written into the simulator, and the model is not validated on real outbreaks.
- Forecasts are weekly, one step ahead and per region; there is no spatial spread between regions and no real-time data feed.
- Strong persistence in the simulation leaves limited room for the models to outperform the baseline.
- Models are trained with default or lightly tuned hyper-parameters; no extensive search was performed.
- Nothing here is medical or public-health advice.

5.3 Future Work

- Replace the simulation with real surveillance data (for example weekly influenza or dengue counts with local weather) using the same CSV schema.
- Add spatial features (neighbouring-region incidence) and longer forecast horizons.
- Tune the decision threshold for recall and report calibration of predicted probabilities.
- Compare against a classical SIR/SEIR model and add automated tests and a small web interface to the predictor.

5.4 Conclusion

This project delivers six foundational machine-learning tasks and an end-to-end outbreak prediction pipeline, all of which run from a single command and report results that are consistent with the code. The logistic regression model achieves 98.25% accuracy on breast-cancer diagnosis, the random forest outperforms a single tree on heart-disease classification, and the capstone demonstrates a leakage-free, time-aware workflow with honest baseline comparison. The principal lesson is that careful evaluation and transparency about data matter more than headline accuracy figures. With real surveillance data in place of the simulation, the same pipeline could serve as the starting point for a genuine early-warning tool.

References

1. Anderson, R. M., & May, R. M. (1991). *Infectious Diseases of Humans: Dynamics and Control*. Oxford University Press.

2. Chinazzi, M., Davis, J. T., Ajelli, M., Gioannini, C., Litvinova, M., Merler, S., et al. (2020). The effect of travel restrictions on the spread of the 2019 novel coronavirus (COVID-19) outbreak. *Science*, 368(6489), 395-400.

3. Kucharski, A. J., Russell, T. W., Diamond, C., Liu, Y., Edmunds, J., Funk, S., & Eggo, R. M. (2020). Early dynamics of transmission and control of COVID-19: a mathematical modelling study. *The Lancet Infectious Diseases*, 20(5), 553-558.

4. Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., et al. (2011). Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12, 2825-2830.

5. Rajkomar, A., Dean, J., & Kohane, I. (2019). Machine learning in medicine. *New England Journal of Medicine*, 380(14), 1347-1358.

6. Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5-32.

7. Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine. https://archive.ics.uci.edu/ml

Appendix A: Repository Structure

```
AI-ML-Internship/
  requirements.txt
  run_all.py                          # runs every script, prints PASS / FAIL
  AI-ML-Internship-Project-Report.pdf # this report
  task1_data_preprocessing/           # Titanic cleaning
  task2_eda/                          # Titanic EDA
  task3_linear_regression/            # Diabetes regression
  task4_logistic_regression/          # Breast cancer classification
  task5_decision_tree_random_forest/  # Heart disease classification
  task6_knn_classification/           # Iris KNN
  disease_outbreak_prediction/        # generate_data.py, main.py, predict.py,
                                      # data/, models/, outputs/

pip install -r requirements.txt
python run_all.py                     # smoke-test all projects
cd disease_outbreak_prediction
python main.py                        # train + evaluate
python predict.py --example high      # predict for one region
```